

⚡ CQRS + MediatR Pattern

Clean Architecture — Command Query Responsibility Segregation

🔗 CQRS কী?

CQRS মানে Command Query Responsibility Segregation। এই pattern এ Read (Query) এবং Write (Command) operation সম্পূর্ণ আলাদা করা হয়। MediatR হলো সেই library যা এই pattern টা ASP.NET Core এ সহজে implement করতে দেয় — Controller সরাসরি business logic জানে না, শুধু Request পাঠায়।

Type	নাম	কাজ
📖	Query	Data পড়ে — কোনো পরিবর্তন করে না (GET)
✏️	Command	Data পরিবর্তন করে — Create, Update, Delete (POST/PUT/DELETE)
✉️	Handler	Query বা Command এর actual কাজ করে
🔗	MediatR	Request নিয়ে সঠিক Handler এ পাঠায় (Mediator)

📦 Required NuGet Packages

📦 Package	Purpose
MediatR	Core MediatR library — Request/Handler pipeline
MediatR.Extensions.Microsoft.DependencyInjection	ASP.NET Core DI integration (v11 এর আগে)

💡 **Note:** MediatR v12+ এ আলাদা Extension package লাগে না। শুধু Install-Package MediatR দিলেই হয়। Program.cs এ AddMediatR() দিয়ে register করো।

📁 Recommended Folder Structure

```
YourProject/  
├── Controllers/
```

```

|   └─ EmployeeController.cs
|─ Features/
|   └─ Employees/
|       └─ Queries/
|           └─ GetAllEmployees/
|               └─ GetAllEmployeesQuery.cs
|                   └─ GetAllEmployeesHandler.cs
|           └─ GetEmployeeById/
|               └─ GetEmployeeByIdQuery.cs
|                   └─ GetEmployeeByIdHandler.cs
|       └─ Commands/
|           └─ CreateEmployee/
|               └─ CreateEmployeeCommand.cs
|                   └─ CreateEmployeeHandler.cs
|           └─ UpdateEmployee/
|               └─ UpdateEmployeeCommand.cs
|                   └─ UpdateEmployeeHandler.cs
|           └─ DeleteEmployee/
|               └─ DeleteEmployeeCommand.cs
|                   └─ DeleteEmployeeHandler.cs
|─ Models/
|   └─ Employee.cs
|─ DTOs/
|   └─ EmployeeDto.cs
|─ Program.cs

```

🔗 Step 1 — Program.cs এ MediatR Register করো

```

var builder = WebApplication.CreateBuilder(args);

// MediatR register – Assembly scan করে সব Handler খুঁজে নেবে
builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(typeof(Program).Assembly));

// DbContext ও AutoMapper থাকলে:
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));

builder.Services.AddAutoMapper(typeof(Program).Assembly);
builder.Services.AddControllers();

```

🌀 Step 2 — Query তৈরি করো (Read Operation)

Query = শুধু data পড়ে, কোনো পরিবর্তন করে না। IRequest<T> implement করতে হয় — T হলো return type।

2A. GetAllEmployees Query + Handler

```
// GetAllEmployeesQuery.cs
using MediatR;

public class GetAllEmployeesQuery : IRequest<List<EmployeeDto>>
{
    // কোনো parameter নেই – সব employee চাই
}

// -----

// GetAllEmployeesHandler.cs
public class GetAllEmployeesHandler
    : IRequestHandler<GetAllEmployeesQuery, List<EmployeeDto>>
{
    private readonly AppDbContext _context;
    private readonly IMapper _mapper;

    public GetAllEmployeesHandler(AppDbContext context, IMapper mapper)
    {
        _context = context;
        _mapper = mapper;
    }

    public async Task<List<EmployeeDto>> Handle(
        GetAllEmployeesQuery request, CancellationToken cancellationToken)
    {
        var employees = await _context.Employees
            .ToListAsync(cancellationToken);
        return _mapper.Map<List<EmployeeDto>>(employees);
    }
}
```

2B. GetEmployeeById Query + Handler

```
// GetEmployeeByIdQuery.cs
public class GetEmployeeByIdQuery : IRequest<EmployeeDto?>
{
    public int Id { get; set; }

    public GetEmployeeByIdQuery(int id) => Id = id;
}

// -----

// GetEmployeeByIdHandler.cs
public class GetEmployeeByIdHandler
    : IRequestHandler<GetEmployeeByIdQuery, EmployeeDto?>
{
    private readonly AppDbContext _context;
    private readonly IMapper _mapper;

    public GetEmployeeByIdHandler(AppDbContext context, IMapper mapper)
    { _context = context; _mapper = mapper; }

    public async Task<EmployeeDto?> Handle(
        GetEmployeeByIdQuery request, CancellationToken cancellationToken)
    {
        var emp = await _context.Employees
            .FindAsync(new object[] { request.Id }, cancellationToken);
        return emp is null ? null : _mapper.Map<EmployeeDto>(emp);
    }
}
```

🔗 Step 3 — Command তৈরি করো (Write Operation)

Command = data পরিবর্তন করে। Create, Update, Delete — সবই Command।

3A. CreateEmployee Command + Handler

```
// CreateEmployeeCommand.cs
public class CreateEmployeeCommand : IRequest<EmployeeDto>
{

```

```

    public string Name      { get; set; } = string.Empty;
    public string Email     { get; set; } = string.Empty;
    public string Department { get; set; } = string.Empty;
    public decimal Salary   { get; set; }
}

// -----

// CreateEmployeeHandler.cs
public class CreateEmployeeHandler
    : IRequestHandler<CreateEmployeeCommand, EmployeeDto>
{
    private readonly AppDbContext _context;
    private readonly IMapper      _mapper;

    public CreateEmployeeHandler(AppDbContext context, IMapper mapper)
    { _context = context; _mapper = mapper; }

    public async Task<EmployeeDto> Handle(
        CreateEmployeeCommand request, CancellationToken cancellationToken)
    {
        var employee = _mapper.Map<Employee>(request);
        _context.Employees.Add(employee);
        await _context.SaveChangesAsync(cancellationToken);
        return _mapper.Map<EmployeeDto>(employee);
    }
}

```

3B. UpdateEmployee Command + Handler

```

// UpdateEmployeeCommand.cs
public class UpdateEmployeeCommand : IRequest<bool>
{
    public int    Id      { get; set; }
    public string Name    { get; set; } = string.Empty;
    public string Email   { get; set; } = string.Empty;
    public string Department { get; set; } = string.Empty;
}

// -----

// UpdateEmployeeHandler.cs

```

```

public class UpdateEmployeeHandler
    : IRequestHandler<UpdateEmployeeCommand, bool>
{
    private readonly AppDbContext _context;
    private readonly IMapper _mapper;

    public UpdateEmployeeHandler(AppDbContext context, IMapper mapper)
    { _context = context; _mapper = mapper; }

    public async Task<bool> Handle(
        UpdateEmployeeCommand request, CancellationToken cancellationToken)
    {
        var emp = await _context.Employees.FindAsync(request.Id);
        if (emp is null) return false;

        _mapper.Map(request, emp); // AutoMapper দিয়ে update
        await _context.SaveChangesAsync(cancellationToken);
        return true;
    }
}

```

3C. DeleteEmployee Command + Handler

```

// DeleteEmployeeCommand.cs
public class DeleteEmployeeCommand : IRequest<bool>
{
    public int Id { get; set; }
    public DeleteEmployeeCommand(int id) => Id = id;
}

// -----

// DeleteEmployeeHandler.cs
public class DeleteEmployeeHandler
    : IRequestHandler<DeleteEmployeeCommand, bool>
{
    private readonly AppDbContext _context;

    public DeleteEmployeeHandler(AppDbContext context)
    { _context = context; }

    public async Task<bool> Handle(

```

```

        DeleteEmployeeCommand request, CancellationToken cancellationToken)
    {
        var emp = await _context.Employees.FindAsync(request.Id);
        if (emp is null) return false;

        _context.Employees.Remove(emp);
        await _context.SaveChangesAsync(cancellationToken);
        return true;
    }
}

```

🔗 Step 4 — Controller এ IMediator Inject ও Use করো

Controller এর কোনো business logic নেই — শুধু Request পাঠায় আর Response return করে:

```

[ApiController]
[Route("api/[controller]")]
public class EmployeeController : ControllerBase
{
    private readonly IMediator _mediator;

    public EmployeeController(IMediator mediator)
        => _mediator = mediator;

    // — GET All —————
    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var result = await _mediator.Send(new GetAllEmployeesQuery());
        return Ok(result);
    }

    // — GET By Id —————
    [HttpGet("{id}")]
    public async Task<IActionResult> GetById(int id)
    {
        var result = await _mediator.Send(new GetEmployeeByIdQuery(id));
        return result is null ? NotFound() : Ok(result);
    }

    // — POST —————
    [HttpPost]

```

```

public async Task<IActionResult> Create(CreateEmployeeCommand command)
{
    var result = await _mediator.Send(command);
    return CreatedAtAction(nameof(GetById), new { id = result.Id }, result);
}

// — PUT —————
[HttpPut("{id}")]
public async Task<IActionResult> Update(int id, UpdateEmployeeCommand command)
{
    command.Id = id;
    var success = await _mediator.Send(command);
    return success ? NoContent() : NotFound();
}

// — DELETE —————
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var success = await _mediator.Send(new DeleteEmployeeCommand(id));
    return success ? NoContent() : NotFound();
}
}

```

📦 Request Flow — কীভাবে কাজ করে?

#	Who	কী করে
1	HTTP Client	API তে Request পাঠায় → POST /api/employee
2	Controller	_mediator.Send(command) call করে — নিজে কিছু করে না
3	MediatR Pipeline	Request এর type দেখে সঠিক Handler খোঁজে
4	Handler	Actual business logic চালায় (DB, AutoMapper, ইত্যাদি)
5	Handler	Result return করে MediatR কে
6	Controller	Result পেয়ে Ok() / Created() / NotFound() return করে
7	HTTP Client	Response পায়

🔗 Bonus — Pipeline Behavior (Logging / Validation)

MediatR এর Pipeline Behavior দিয়ে প্রতিটা Request এর আগে/পরে কোনো কাজ করা যায় — যেমন logging, validation। এটা Middleware এর মতো কিন্তু শুধু MediatR এর জন্য:

```
// LoggingBehavior.cs
public class LoggingBehavior<TRequest, TResponse>
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : IRequest<TResponse>
{
    private readonly ILogger<LoggingBehavior<TRequest, TResponse>> _logger;

    public LoggingBehavior(ILogger<LoggingBehavior<TRequest, TResponse>> logger)
        => _logger = logger;

    public async Task<TResponse> Handle(
        TRequest request,
        RequestHandlerDelegate<TResponse> next,
        CancellationToken cancellationToken)
    {
        _logger.LogInformation("Handling {Name}", typeof(TRequest).Name);

        var response = await next();    // ← actual Handler call হয়

        _logger.LogInformation("Handled {Name}", typeof(TRequest).Name);
        return response;
    }
}

// Program.cs এ register:
builder.Services.AddTransient(
    typeof(IPipelineBehavior<,>),
    typeof(LoggingBehavior<,>));
```

Quick Reference

Interface / Method	কাজ
IRequest<T>	Query বা Command class — T হলো return type
IRequestHandler<TReq, TRes>	Handler class — Handle() method implement করে
IRequest (without T)	Return type ছাড়া Command (void এর মতো)
_mediator.Send(request)	Request পাঠায় ও response await করে

<code>_mediator.Publish(notification)</code>	Event publish করে (multiple handler চলে)
<code>INotification</code>	Event/Notification class এর interface
<code>INotificationHandler<T></code>	Event handler — একাধিক থাকতে পারে
<code>IPipelineBehavior<TReq, TRes></code>	Middleware-like — logging, validation এর জন্য
<code>AddMediatR(cfg => ...)</code>	Program.cs এ সব Handler register করে

☑ **মনে রেখো:** Controller শুধু mediator.Send() করবে — কোনো business logic রাখবে না। সব logic Handler এ থাকবে। এটাই CQRS + MediatR এর মূল কথা।

⚠ **Common Mistake:** Handler কে manually new করবে না। AddMediatR() register করলে DI Container নিজেই Handler inject করবে।